Niord Message Presentation

Table of Contents

1. Introduction 1
2. Presentation 2
    2.1. Translations 2
    2.2. Message Lists 3
    2.3. Message Details 4
        2.3.1. The HTML of a message 6
    2.4. Message Geometry 7

Chapter 1. Introduction

Niord is a system for writing, publishing and archiving Navigational Warnings (NW) and Notices to Mariners (NM). It is developed and run by the Danish Emergency Management Agency.

This document gives guidelines for how a client, such as a navigation system, a website or an app, can show Niord's messages to a reader. It assumes the messages are in the Niord Message Model format, as the Niord Public API returns them; please read those two documents first.

The example throughout is the public site that comes with Niord, a client of the public API that shows the messages in force to mariners in three views: the full text of each message, a list of titles, and a map. One deployment of it can be seen at nautiskinformation.brs2.dk. In the source code, which is published under https://github.com/NiordOrg, the public site is the folder `fe/ui-apps/info`, and the rules below point to the parts of it that follow them.

Chapter 2. Presentation

The Niord message model is rich, and only some of it is meant for the reader. Some fields help a client sort, filter or draw messages without being shown themselves:

- *Message.areas* is usually not shown as such, since the title already names the areas, but it is useful for grouping and filtering messages.
- *Message.categories* is usually not shown either, but may decide how a message is drawn, for example with a special symbol for firing exercises.
- *MessagePart.eventDates* is for filtering and calculating; the details text of the part states the dates in words.
- *Message.followUpDate* is an aid for the editors and is never shown.
- *MessagePart.indexNo* carries no information; the parts come in the order of the list.

2.1. Translations

As the Niord Message Model describes, every class with texts that depend on the language has a list of description entries, one per language. A *Message*, for example, has a list of *MessageDesc* entries:

```
{
  "descs": [
    {
      "lang": "en",
      "title": "Denmark. The Baltic Sea. Rønne. Vesthavnen. Construction works.",
      "vicinity": "Rønne. Vesthavnen."
    },
    {
      "lang": "da",
      "title": "...",
      "vicinity": "..."
    }
  ]
}
```

When the request to the public API names no language, every language is returned, as above, and the client looks through each `descs` list for the entry in the reader's language.

When the request names a language with the `lang` parameter, each list holds one entry, and the client can take the first one (`descs[0]`). If the message has no text in the language asked for, the entry is another language the message has, and its `lang` says which. The client should then show that the text is in another language: the public site puts a small flag of that language after such a title.

The public site asks for the reader's language and still sorts every `descs` list so that the reader's language comes first, so that the rest of its code can always take the first entry.

2.2. Message Lists

When messages are shown as a list, the short id and the title of each message are usually enough. In simplified pseudo-code, for a client that writes HTML:

```
for msg in message-list
  emit-title-line(msg)
end-for

procedure emit-title-line(msg)
  emit "<div>"

  /** The short id, or the type of a message without one **/
  if msg.shortId defined
    emit "<span>" + msg.shortId + "</span>"
  else
    emit "<span>" + label-of(msg.type) + " " + msg.mainType + "</span>"
  end-if

  /** Mark a message that is no longer in force **/
  if msg.status == "CANCELLED"
    emit " <span>Cancelled</span>"
  else if msg.status == "EXPIRED" and msg.type != "PERMANENT_NOTICE"
    emit " <span>Expired</span>"
  end-if

  /** For a temporary or preliminary notice, emit (T) or (P) **/
  if msg.type == "TEMPORARY_NOTICE"
    emit " (T)"
  else if msg.type == "PRELIMINARY_NOTICE"
    emit " (P)"
  end-if

  /** The title **/
  emit " <span>" + msg.descs[0].title + "</span>"

  emit "</div>"
end-procedure
```

A search in the public API returns messages that are in force, so a client that lists only those never sees the status marks. A list of the messages of a publication, such as an old weekly Notices to Mariners, may hold messages that have since been cancelled or have expired; the public site marks them, but not an expired permanent notice, whose expiry is the ordinary end of its life.

The public site also groups the list under area headings. It takes the second level of the first area of each message (for example *The Baltic Sea* under *Denmark*) and writes a heading whenever it changes from one message to the next. The API usually returns the messages in the order of the area tree, so each area then gets one heading.

The list then looks like this on the public site:



1. Message List

2.3. Message Details

When one message, *msg*, is shown in full, more of its fields are of interest. In simplified pseudo-code, in the order the public site shows them:

```
/** Pictures the editor placed above the message **/
for att in msg.attachments where att.display == "ABOVE"
  emit-attachment(att)
end-for

/** A star when the message is based on original information **/
if msg.originalInformation == true
  emit "<div>*</div>"
end-if

/** The title line, as in the list **/
emit-title-line(msg)

/** References to other messages **/
for ref in msg.references
  emit "<div>References: " + ref.messageId + " " + label-of(ref.type)
  if ref.descs[0].description defined
    emit " - " + ref.descs[0].description
  end-if
  emit "</div>"
end-for

/** The message parts **/
for part in msg.parts
```

```
  /**
   * part.type is one of DETAILS, TIME, POSITIONS, NOTE, PROHIBITION or SIGNALS,
   * and serves as a heading: "Details", "Time", ... The public site writes the
   * heading only when the type differs from that of the part above.
   */
  emit "<div>" + label-of(part.type) + ":"
  if part.descs[0].subject defined and part.hideSubject != true
    emit "<strong>" + part.descs[0].subject + "</strong>"
  end-if
  emit "<div class='message-details-html'>" + clean(part.descs[0].details) + "</div>"
  emit "</div>"
end-for

/**
 * Attachments. The public site shows this list only after the reader
 * presses "Show attachments".
 */
for att in msg.attachments
  emit "<div>Attachments: <a href='" + att.path + "'>" + att.fileName + "</a>"
  if att.descs[0].caption defined
    emit " - " + att.descs[0].caption
  end-if
  emit "</div>"
end-for

/** Charts, for example "Charts: 189 (INT 1336), 188." **/
if msg.charts defined and not empty
  chartTexts = []
  for chart in msg.charts
    text = chart.chartNumber
    if chart.internationalNumber defined
      text = text + " (INT " + chart.internationalNumber + ")"
    end-if
    chartTexts.add(text)
  end-for
  emit "<div>Charts: " + join(chartTexts, ", ") + ".</div>"
end-if

/** Publication **/
if msg.descs[0].publication defined
  emit "<div>Publication: " + clean(msg.descs[0].publication) + "</div>"
end-if

/**
 * Source and publish date, for example
 * "(Port authority 18 August 2026. Published 19 August 2026)"
 */
if msg.descs[0].source defined or msg.publishDateFrom defined
  text = ""
  if msg.descs[0].source defined
    text = msg.descs[0].source
```

```
  end-if
  if msg.publishDateFrom defined
    if text not empty
      if text does not end with "."
        text = text + "."
      end-if
      text = text + " "
    end-if
    text = text + "Published " + format-date(msg.publishDateFrom)
  end-if
  emit "<div style='text-align:right'>(" + text + ")</div>"
end-if

/** Pictures the editor placed below the message **/
for att in msg.attachments where att.display == "BELOW"
  emit-attachment(att)
end-for
```

`emit-attachment` shows a picture (or a video, when the attachment's `type` starts with `video`) with its caption, in the size the attachment's `width` and `height` give, but never wider than the space there is.

The client must be able to show the HTML of the message part details and of the publication field. A client that cannot show HTML should convert it to plain text first.

2.3.1. The HTML of a message

The HTML of a message part holds paragraphs, lists, tables, links and pictures. Niord's own applications clean it before they show it (in the source code, `fe/shared-html/`), and a client that shows the HTML in a web page should do the same, since the text comes from another system:

- Elements and attributes that can run code (`<script>`, event attributes such as `onclick`, `javascript:` links) are removed.
- Every class except the ones in the table below is removed. A client should style these classes and needs no others.
- Of the inline styles (`style` attributes), only `text-align` (`left`, `right`, `center` or `justify`) and `padding-left` (a length of at most 160 pixels) are kept. Every other inline style is left over from text pasted from a word processor, and is dropped.
- A link that opens a new window (`target`) is given `rel="noopener noreferrer"`.

Class	What it marks, and how Niord's applications show it
position-table	A table of positions, as Niord writes them in the details of a message: one row per position, with a `pos-index` cell and a `pos-col` cell.
pos-index	The number of a position in a position table, such as `1)`. Aligned to the right, never wrapped.
pos-col	The coordinates of a position in a position table, such as `55° 05.887'N - 014° 41.396'E`. Aligned to the right, never wrapped.

Class	What it marks, and how Niord's applications show it
positions	A compact list of positions in a table: no borders, no wrapping, and a first column aligned to the right in a lighter colour.
no-border	A table without borders.
condensed	A table whose cells have no padding.
underline	A table row or cell with a line under it.
no-wrap	A piece of text that must not be broken across lines.

The publication field may hold links to publications with the publication's id in an attribute of their own (see Publication citations). The public site shows these links in the colour of the text around them, and underlines them when the pointer is over them, so that they read as part of the citation.

The public site shows the message details like this:



2. Message Details

2.4. Message Geometry

A client may want to show messages on a map, either all the messages of a list or the one message the reader is looking at.

Since the geometry of a message part is GeoJSON, most map libraries can draw it with little work. As the Niord Message Model describes, Niord adds properties to each GeoJSON feature, such as names for the feature and for its positions.

For a message, *msg*, to be drawn with the map library OpenLayers, the code may look like this in simplified pseudo-code:

```
import GeoJSON from "ol/format/GeoJSON"

geoJsonFormat = new GeoJSON()
layer = ...

/** Draw the features of every message part **/
for part in msg.parts
  /** part.geometry is always a FeatureCollection **/
```

```
  if part.geometry defined
    for feature in part.geometry.features
      olFeature = geoJsonFormat.readFeature(feature, {
                    dataProjection: "EPSG:4326",
                    featureProjection: "EPSG:3857"
                  })
      olFeature.set("message", msg)
      layer.getSource().addFeature(olFeature)
    end-for
  end-if
end-for
```

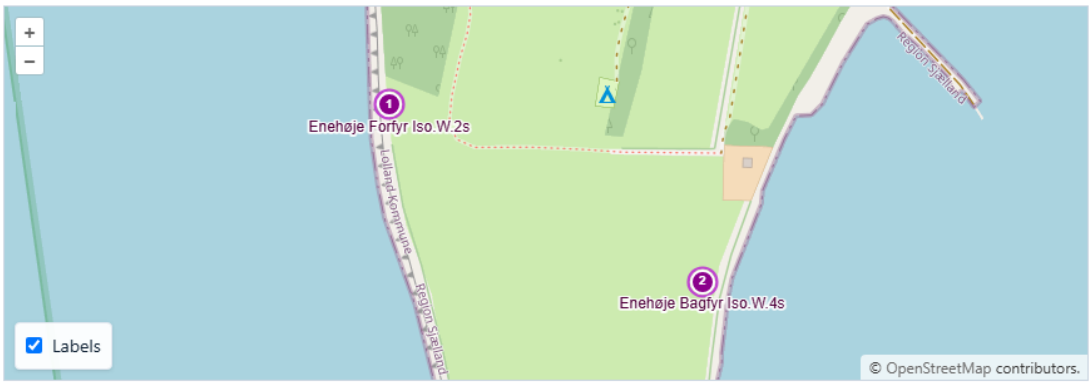
The client should also handle a message without a geometry. The public site lists such messages in a panel that lies over the map, in its top right corner (across the top on a phone), and that the reader can minimize.

On the map of a single message, the public site numbers the positions the way the position tables in the text do, and writes the name of each position beside it:

- It numbers the positions from 1, across all the parts and features of the message, in the order the coordinates come. The `startCoordIndex` property of a feature holds the number its first position had when the editor numbered the positions, which a client may use instead.
- It takes the name of a position from the property `name:<x>:<lang>` of the feature, where `<x>` is the place of the position in the feature's coordinates, counted from 0, and the name of the whole feature from `name:<lang>`.
- It does not number the closing position of a polygon, the positions of a hole in a polygon, or the positions of a feature that is an affected area (`"restriction": "affected"`) or that was computed from other features (it has `parentFeatureIds`). Those areas are drawn, but they are not positions the message gives.

The map of a single message on the public site looks like this:

Notices to Mariners



★

NM-811-26 (T)

Denmark. The Great Belt. Nakskov Fjord. Enehøje. Light unlit.

References NM-688-26 (repetition with new time)

Details **Until ending September 2026,** the lights in positions 1) and 2) are unlit.

1)	54° 50.318'N - 011° 00.882'E	Enehøje Forfyr Iso.W.2s
2)	54° 50.213'N - 011° 01.204'E	Enehøje Bagfyr Iso.W.4s

Charts 144, 142 (INT 1368), 103 (INT 1303).

Publication Danish List of Lights (Dansk Fyrliste) no C1718 + C1718.1.

(*DEMA 15 July and Lolland Havne 25 August 2026. Published 26 August 2026*)

Close

3. Message Geometry